

Mini Project - ii ShopEZ - E-Commerce Application

Backend Technical Documentation

Version 1.0 | .NET 10 | ASP.NET Core Web API

1. Project Overview

The shopEZ has enhanced RESTful backend service built with ASP.NET Core (.NET 10). It a complete e-commerce backend with user authentication, product catalog management, and order processing. The API uses JWT-based authentication with role-based access control (Admin / User), and is fully documented via Swagger UI.

What this API does

Allows users to register and log in, browse and manage products, place and track orders — all secured with JWT tokens and role-based authorization.

2. Tech Stack

Layer	Technology	Version
Framework	ASP.NET Core Web API	.NET 10
Language	C#	C# 13
ORM	Entity Framework Core	10.0.5
Database	Microsoft SQL Server	SQL Server Express
Authentication	JWT Bearer Tokens	—
Password Hashing	BCrypt	BCrypt.Net-Next 4.1.0
API Documentation	Swagger / Swashbuckle	6.5.0
Architecture Pattern	Repository + Service Pattern	—

3. Project Structure

The project follows a clean layered architecture with clear separation of concerns:

Folder / File	Purpose
Controllers/	HTTP request handlers — Auth, Products, Orders
Services/	Business logic layer — implements interfaces
Repositories/	Data access layer — EF Core queries

Models/	Database entity classes + AppDbContext
DTOs/	Data Transfer Objects for request/response shaping
Migrations/	EF Core database migration history
Program.cs	Application entry point, DI registration, middleware setup
appsettings.json	Configuration — DB connection string, JWT settings
launchSettings.json	Local dev run profiles and port configuration
WebApplication17.csproj	Project definition and NuGet package references

4. Dependencies (NuGet Packages)

Package	Version	Purpose
Microsoft.AspNetCore.Authentication.JwtBearer	10.0.5	JWT token authentication middleware
Microsoft.EntityFrameworkCore	10.0.5	ORM core library
Microsoft.EntityFrameworkCore.SqlServer	10.0.5	SQL Server EF provider
Microsoft.EntityFrameworkCore.Tools	10.0.5	Migrations CLI tools (dev only)
Swashbuckle.AspNetCore	6.5.0	Swagger/OpenAPI documentation UI
System.IdentityModel.Tokens.Jwt	8.17.0	JWT token creation and validation
BCrypt.Net-Next	4.1.0	Secure password hashing and verification

5. Database Models

The application uses 4 tables managed by Entity Framework Core with SQL Server.

5.1 User

Column	Type	Constraints	Description
UserId	int	PK, Identity	Auto-increment primary key
Name	string	Required	Full name of the user

Email	string	Required, Unique	User email — unique index enforced
Password	string	Required	BCrypt hashed password (never plain text)
Role	string	Required	Either 'Admin' or 'User'

5.2 Product

Column	Type	Constraints	Description
ProductId	int	PK, Identity	Auto-increment primary key
Name	string	Required	Product name
Description	string	Required	Product description
Price	decimal(18,2)	Range: 1–100,000	Unit price
ImageUrl	string	Required	URL to product image
Stock	int	Range: 0–MaxInt	Available inventory count

5.3 Order

Column	Type	Constraints	Description
OrderId	int	PK, Identity	Auto-increment primary key
UserId	int	FK → Users	Reference to the user who placed the order
OrderDate	DateTime	Auto-set (UTC)	Timestamp when order was created
TotalAmount	decimal(18,2)	Computed	Sum of (Price × Quantity) for all items

5.4 OrderItem

Column	Type	Constraints	Description
OrderItemId	int	PK, Identity	Auto-increment primary key
OrderId	int	FK → Orders (Cascade)	Parent order — deleted with order

ProductId	int	FK → Products (Restrict)	Product reference — cannot delete if in order
Quantity	int	Range: 1–100	Number of units ordered
Price	decimal(18,2)	Snapshot	Price at time of order (not live product price)

5.5 Entity Relationships

- User → Orders: One-to-Many (one user can have many orders). Cascade delete.
- Order → OrderItems: One-to-Many (one order has many line items). Cascade delete.
- Product → OrderItems: One-to-Many (a product can appear in many orders). Restrict delete.

6. How It Works

6.1 Architecture Flow

Every request flows through these layers in order:

Step	Layer	Responsibility
1	Controller	Receives HTTP request, validates input, extracts JWT claims
2	Service	Applies business logic, validation rules, data transformation
3	Repository	Executes EF Core queries against SQL Server
4	Database	Returns data — flows back up through Service → Controller → HTTP Response

6.2 Authentication Flow

- User registers via POST /api/auth/register — password is BCrypt hashed before saving.
- User logs in via POST /api/auth/login — BCrypt.Verify() checks the hash.
- On successful login, a JWT token is generated containing: UserId, Name, Email, Role.
- Token expires after 2 hours.
- All protected endpoints require the header: Authorization: Bearer <token>
- Role-based access: Admin-only endpoints are decorated with [Authorize(Roles = "Admin")].

6.3 Order Processing Flow

- User sends POST /api/orders with a list of { ProductId, Quantity } items.

- UserId is automatically extracted from the JWT token (not from the request body).
- Repository validates: user exists, all products exist, quantities are valid.
- Price is fetched from the live product record (not from the client request).
- TotalAmount = sum of (product.Price × item.Quantity) for all items.
- Order and OrderItems are saved in a single SaveChangesAsync() call.

6.4 Role-Based Access Summary

Feature	User (logged in)	Admin
Register / Login	✓ Public	✓ Public
View all products	✓ Yes	✓ Yes
View product by ID	✓ Yes	✓ Yes
Create product	✗ No	✓ Yes
Update product	✗ No	✓ Yes
Delete product	✗ No	✓ Yes
Create order	✓ Own orders only	✓ Yes
View orders	✓ Own orders only	✓ All orders
View order by ID	✓ Own orders only	✓ Any order
Update order	✗ No	✓ Yes
Delete order	✗ No	✓ Yes

6.5 Dependency Injection Setup

All services and repositories are registered with Scoped lifetime in Program.cs:

```
builder.Services.AddScoped<IOrderRepository, OrderRepository>();
```

```
builder.Services.AddScoped<IOrderService, OrderService>();
builder.Services.AddScoped<IProductRepository, ProductRepository>();
```

```
builder.Services.AddScoped<IProductService, ProductService>();
```

```
builder.Services.AddScoped<IAuthService, AuthService>();
```

7. Configuration

7.1 appsettings.json

Key	Value	Des
ConnectionStrings.DefaultConnection	Server=...\SQLEXPRESS;Database=ProjectDB;...	SQL conn strin
Jwt.Key	ThisIsAVeryLongSuperSecureSecretKeyForJwtToken12345	Secr for JWT
Jwt.Issuer	MyApp	JWT issu
Jwt.Audience	MyAppUsers	JWT audi
Logging.LogLevel.Default	Information	Defa leve

Security Note

The JWT Key in appsettings.json should be replaced with a strong secret and stored in environment variables or a secrets manager before deploying to production. Never commit real secrets to source control.

7.2 launchSettings.json — Run Profiles

Profile	URL	Environment
http	http://localhost:5293	Development
https	https://localhost:7138 / http://localhost:5293	Development

Swagger UI launches automatically at the root URL (http://localhost:5293) when running in Development mode.

8. API Endpoints

Base URL (Development): http://localhost:5293 | https://localhost:7138

Swagger UI: http://localhost:5293 (auto-redirects from /swagger)

8.1 Authentication — /api/auth

[POST] /api/auth/register

Auth: Public — no token required

Register a new user account.

Register

Request Body:

Field	Type	Required	Description
name	string	Yes	Full name of the user
email	string	Yes	Valid email address (must be unique)
password	string	Yes	Plain text password — hashed by server
role	string	Yes	Either 'Admin' or 'User'

Responses:

- 200 OK — "The user registered successfully"
- 400 Bad Request — "This Record already exists" (duplicate email)
- 400 Bad Request — email or password missing

Login

[POST] /api/auth/login

Auth: Public — no token required

Log in and receive a JWT token.

Request Body:

Field	Type	Required	Description
email	string	Yes	Registered user email
password	string	Yes	Account password

Response Body (200 OK):

```
{ "token": "<JWT_TOKEN>", "user": { "userId": 1, "name": "John", "email": "john@example.com", "role": "User" } }
```

- 401 Unauthorized — invalid email or password
- 500 Internal Server Error — unexpected error

8.2 Products — /api/products

Get All Products

[GET] /api/products

Auth: JWT Required — Any authenticated user

Returns a list of all products in the catalog.

- 200 OK — Array of product objects
- 401 Unauthorized — no or invalid token

Get Product by ID

[GET] /api/products/{id}

Auth: JWT Required — Any authenticated user

Returns a single product by its ID.

- 200 OK — Product object
- 400 Bad Request — invalid ID (≤ 0)
- 404 Not Found — product does not exist

Create Product

[POST] /api/products

Auth: JWT Required — Admin only

Create a new product. Requires Admin role.

Request Body:

Field	Type	Required	Validation
name	string	Yes	Cannot be empty

description	string	Yes	Cannot be empty
price	decimal	Yes	Range: 1 – 100,000
imageUrl	string	Yes	Cannot be empty
stock	int	Yes	Range: 0 – MaxInt

- 200 OK — "Product created successfully...!!"
- 400 Bad Request — validation errors
- 401 Unauthorized / 403 Forbidden — not Admin

Update Product

[PUT] /api/products/{id}

Auth: JWT Required — Admin only

Update an existing product by ID.

- 200 OK — Updated successfully
- 400 Bad Request — invalid ID or validation errors
- 404 Not Found — product not found

Delete Product

[DELETE] /api/products/{id}

Auth: JWT Required — Admin only

Delete a product by ID. Cannot delete if product exists in any order.

- 200 OK — Deleted successfully
- 400 Bad Request — invalid ID
- 404 Not Found — product not found

8.3 Orders — /api/orders

All order endpoints require a valid JWT token. Users can only see/modify their own orders. Admins can access all orders.

Create Order

[POST] /api/orders

Auth: JWT Required — Any authenticated user

Place a new order. UserId is taken from the JWT token automatically.

Request Body:

Field	Type	Required	Description
items	array	Yes	List of order items
items[].productId	int	Yes	ID of the product to order
items[].quantity	int	Yes	Quantity (range: 1–100)

Response Body (200 OK):

```
{ "orderId": 5, "userId": 1, "orderDate": "2026-04-14T...", "totalAmount": 149.99, "items": [ {
  "productId": 2, "productName": "Widget", "quantity": 3, "price": 49.99 } ] }
```

- 400 Bad Request — empty items, invalid product IDs, quantity issues
- 500 Internal Server Error — unexpected errors

Get All Orders

[GET] /api/orders

Auth: JWT Required — Users see own; Admins see all

Returns orders. Regular users only receive their own orders filtered by UserId from JWT.

- 200 OK — Array of order response objects

Get Order by ID

[GET] /api/orders/{id}

Auth: JWT Required — Own order (User) or any (Admin)

Returns a single order. Users get 403 Forbidden if they try to access another user's order.

- 200 OK — Order object
- 403 Forbidden — user trying to access another user's order
- 404 Not Found — order does not exist

Update Order

[PUT] /api/orders/{id}

Auth: JWT Required — Admin only

Replace all order items and recalculate TotalAmount. OrderDate is updated to current UTC time.

- 200 OK — Order updated successfully
- 404 Not Found — order not found

Delete Order

[DELETE] /api/orders/{id}

Auth: JWT Required — Admin only

Permanently delete an order and all its items (cascade delete).

- 200 OK — Order deleted successfully
- 404 Not Found — order not found

9. Middleware Pipeline

Requests pass through these middleware layers in this exact order:

Order	Middleware	Purpose
1	Swagger / SwaggerUI	Serves API documentation at root URL (development)
2	HTTPS Redirection	Redirects HTTP requests to HTTPS
3	CORS (AllowAll policy)	Allows requests from any origin, method, and header
4	Authentication	Validates JWT Bearer tokens on incoming requests
5	Authorization	Enforces [Authorize] and [Authorize(Roles=...)] attributes
6	MapControllers	Routes requests to the appropriate controller actions

10. Setup & Installation

Prerequisites

- .NET 10 SDK installed
- SQL Server Express (or full SQL Server) installed and running
- Visual Studio 2022+ or VS Code with C# extension

Step-by-Step Setup

Step	Action	Command / Detail
1	Clone or open the project	Open WebApplication17.sln in Visual Studio

2	Update connection string	Edit appsettings.json → ConnectionStrings.DefaultConnection
3	Update JWT key	Edit appsettings.json → Jwt.Key (use a strong secret)
4	Restore NuGet packages	dotnet restore (auto in VS)
5	Apply database migrations	dotnet ef database update (in Package Manager Console)
6	Run the application	Press F5 in VS or run: dotnet run
7	Open Swagger UI	Navigate to http://localhost:5293

Testing with Swagger

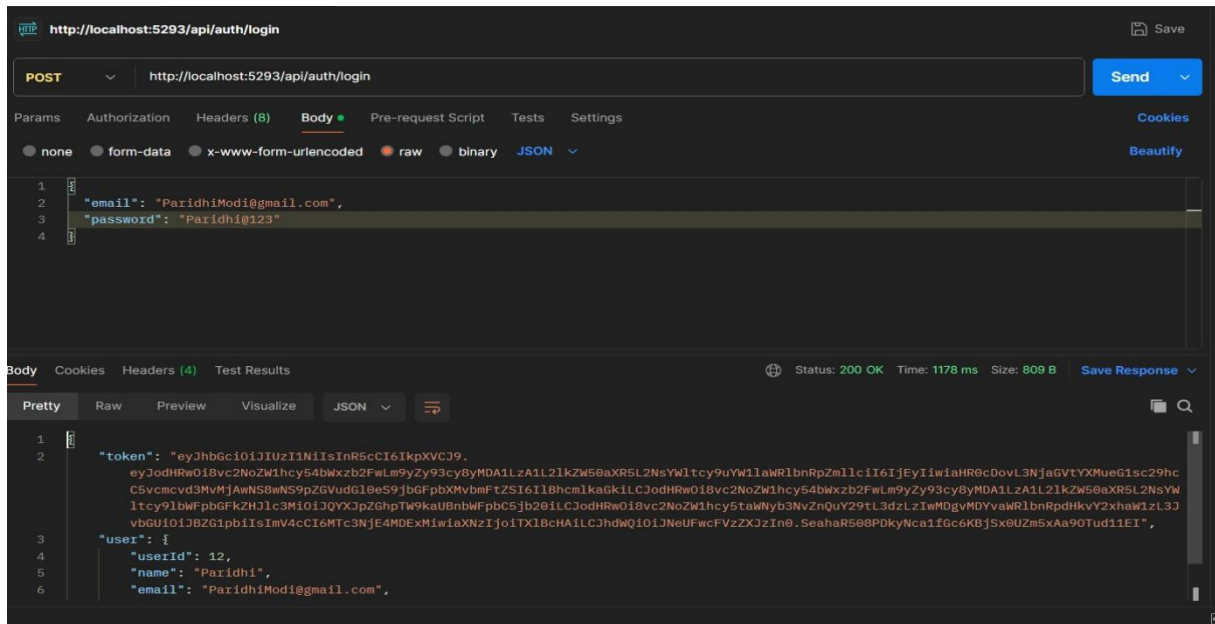
- Open http://localhost:5293 — Swagger UI loads automatically.
- Call POST /api/auth/register to create a user with role 'Admin' or 'User'.
- Call POST /api/auth/login to get a JWT token.
- Click Authorize (lock icon) in Swagger and enter: Bearer <your_token>
- All protected endpoints are now accessible.

11. HTTP Status Code Reference

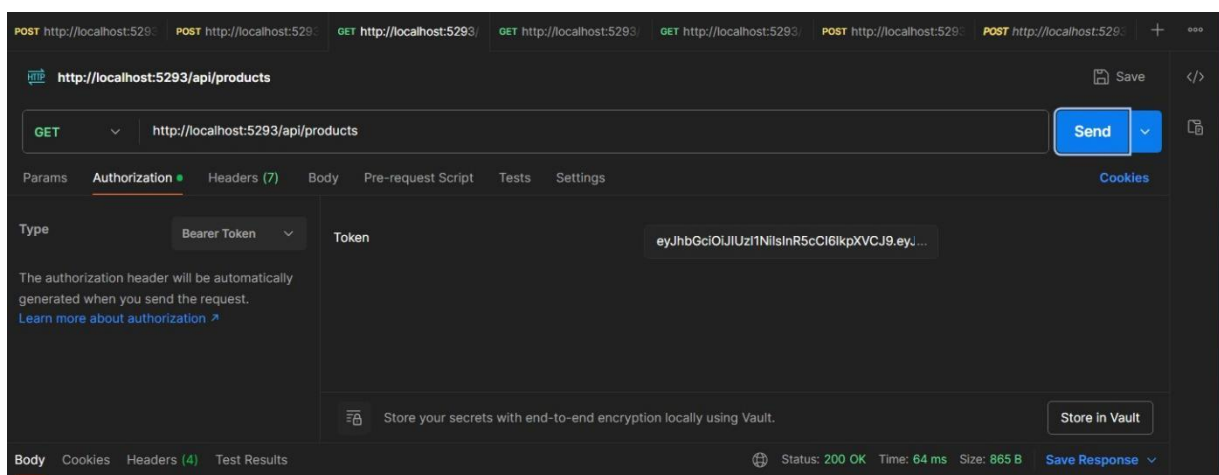
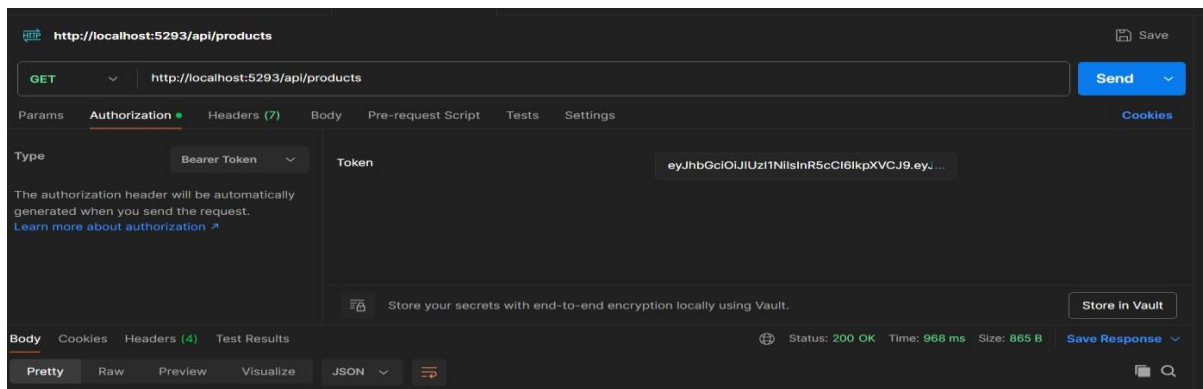
Code	Status	When it occurs
200	OK	Request succeeded — returns data or success message
400	Bad Request	Invalid input, missing fields, validation failures, duplicate email
401	Unauthorized	No token provided or token is invalid/expired
403	Forbidden	Valid token but insufficient role (e.g., User accessing Admin route)
404	Not Found	Resource with given ID does not exist
500	Internal Server Error	Unhandled exception on the server

API TESTING – TEST SCENARIOS

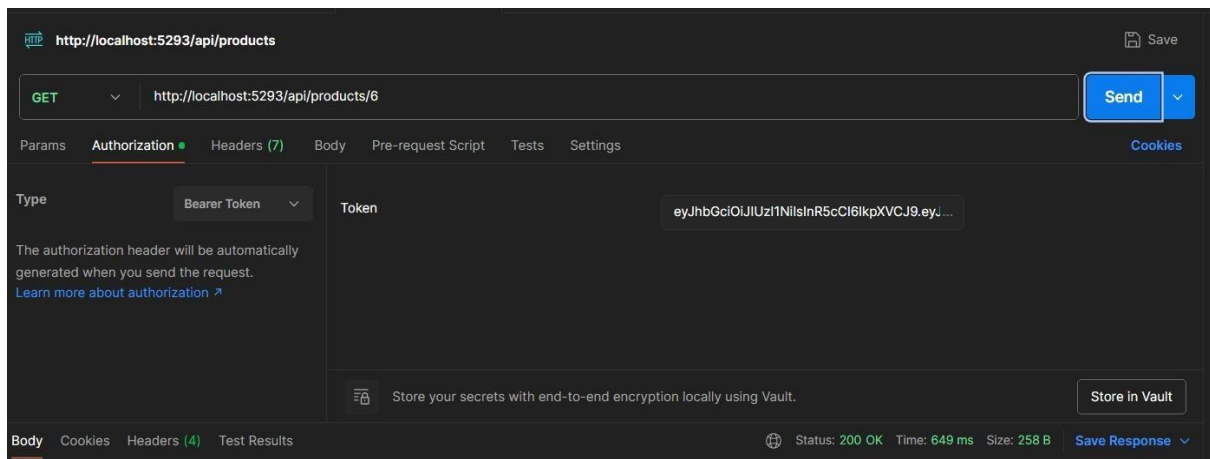
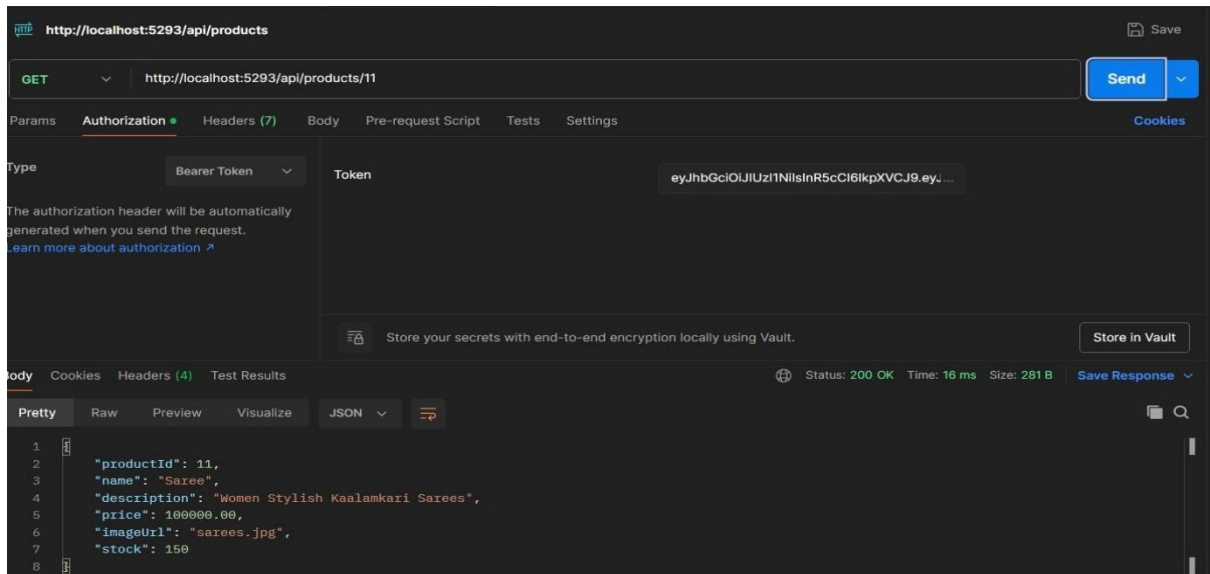
1. User and Admin Register



3. View All Products – Admin and user both can access

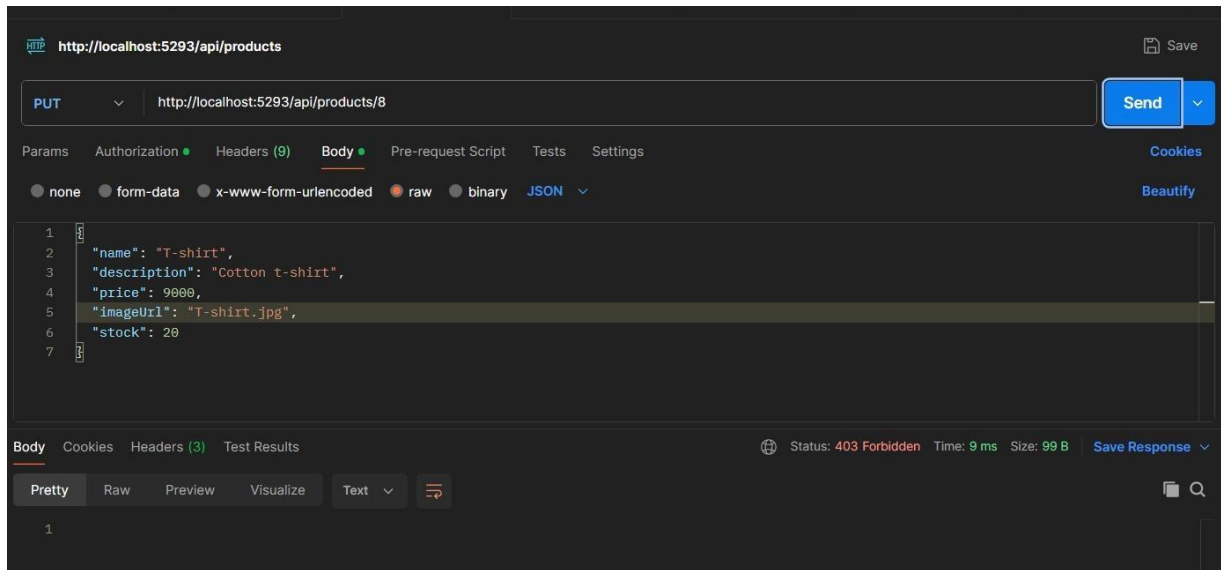


4. Get Product by Id- Admin and user can access

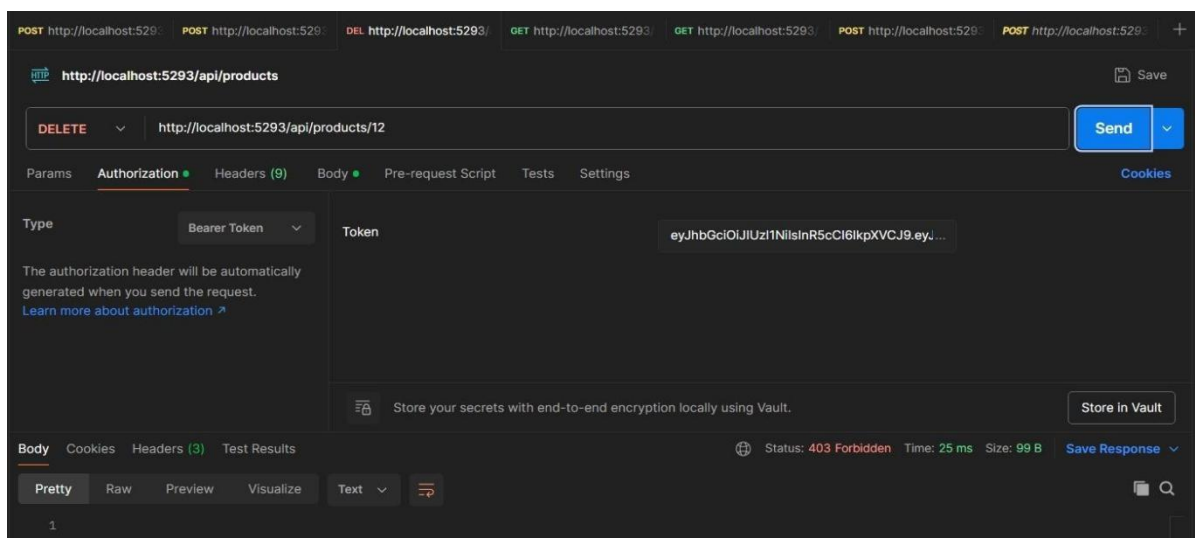
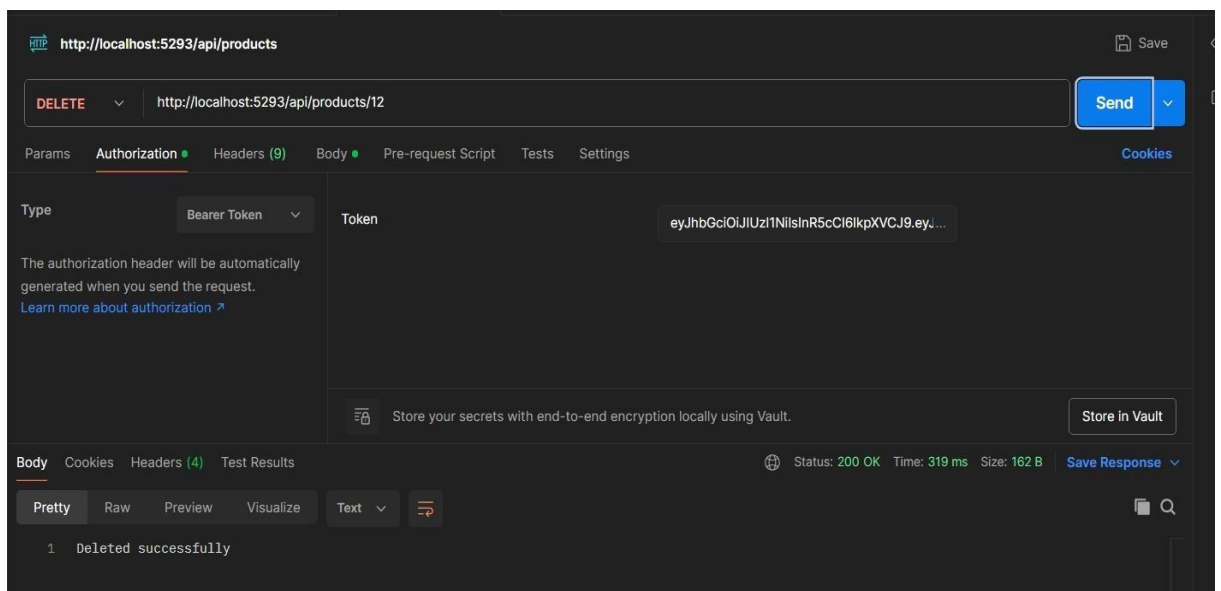


```
{  "productId": 6,  "name": "Shoes",  "description": "Running Shoes",  "price": 2000.00,  "imageUrl": "shoes.jpg",  "stock": 15},
```

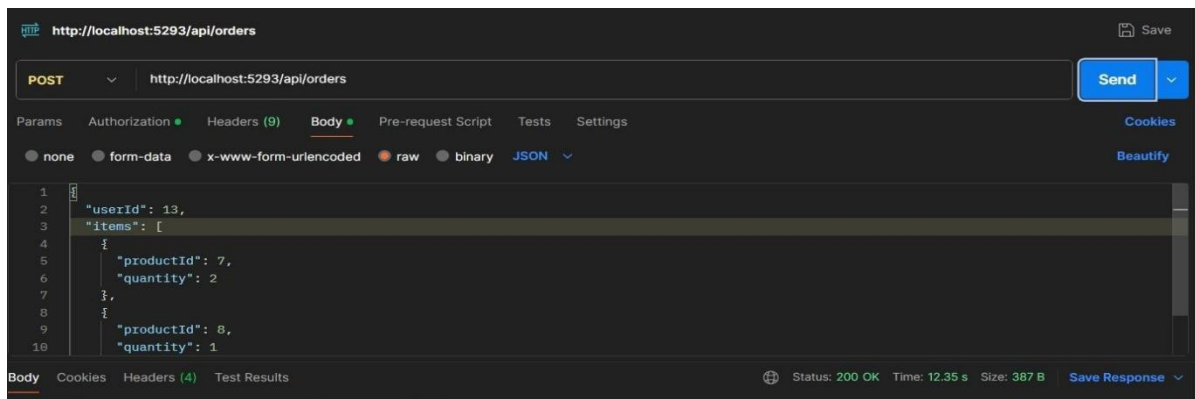
5. Create Product Admin only Access



7.Delete Admin Only Access



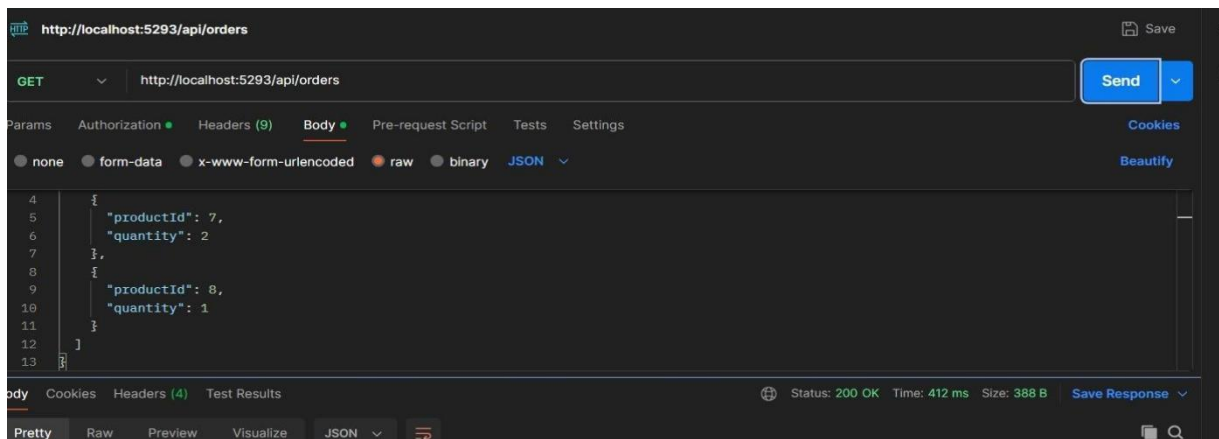
8. Create orders Admin can view all orders and user can view own orders



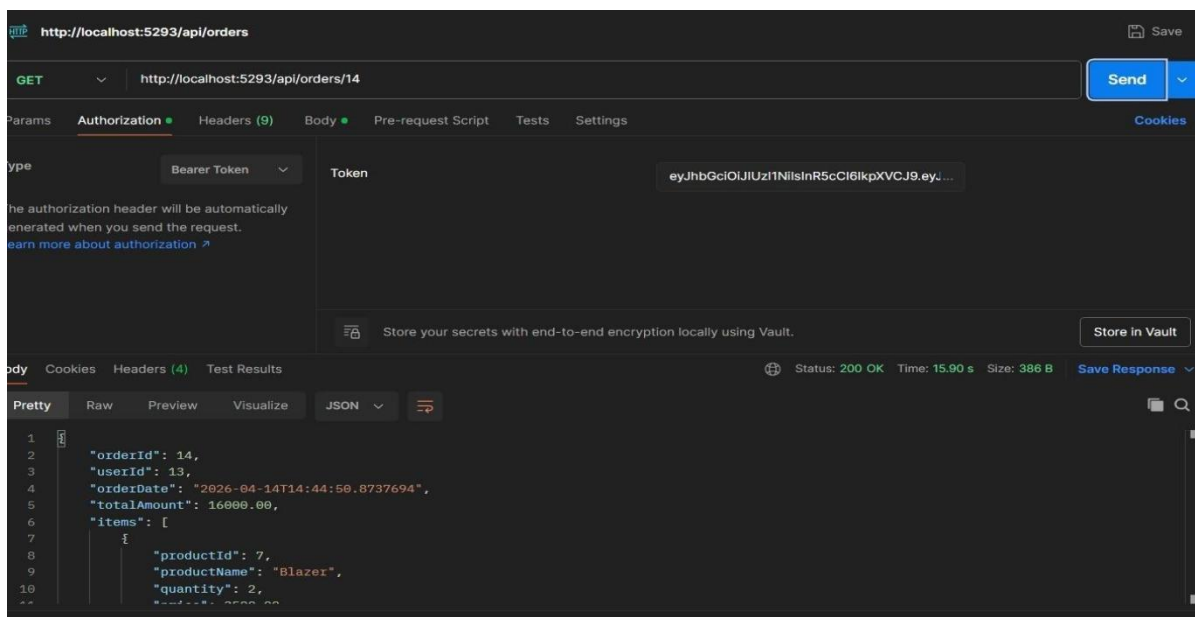
```
[
  {
    "productId": 6,
    "name": "Shoes",
    "description": "Running Shoes",
    "price": 2000.00,
    "imageUrl": "shoes.jpg",
    "stock": 15
  },
  {
    "productId": 7,
    "name": "Blazer",
    "description": "Formal Blazer",
    "price": 3500.00,
    "imageUrl": "blazer.jpg",
    "stock": 8
  },
  {
    "productId": 8,
    "name": "T-Shirt",
    "description": "Cotton T-Shirt",
    "price": 9000.00,
    "imageUrl": "tshirt.jpg",
    "stock": 15
  }
]
```

```
},  
{  
  "productId": 9,  
  "name": "Watches",  
  "description": "Men's Watch",  
  "price": 75000.00,  
  "imageUrl": "watch.jpg",  
  "stock": 15  
},  
{  
  "productId": 10,  
  "name": "Sunglassess",  
  "description": "Men's Stylish Sunglass",  
  "price": 1500.00,  
  "imageUrl": "sunglasses.jpg",  
  "stock": 15  
},  
{  
  "productId": 11,  
  "name": "Saree",  
  "description": "Women Stylish Kaalamkari Sarees",  
  "price": 100000.00,  
  "imageUrl": "sarees.jpg",  
  "stock": 150  
}  
]
```

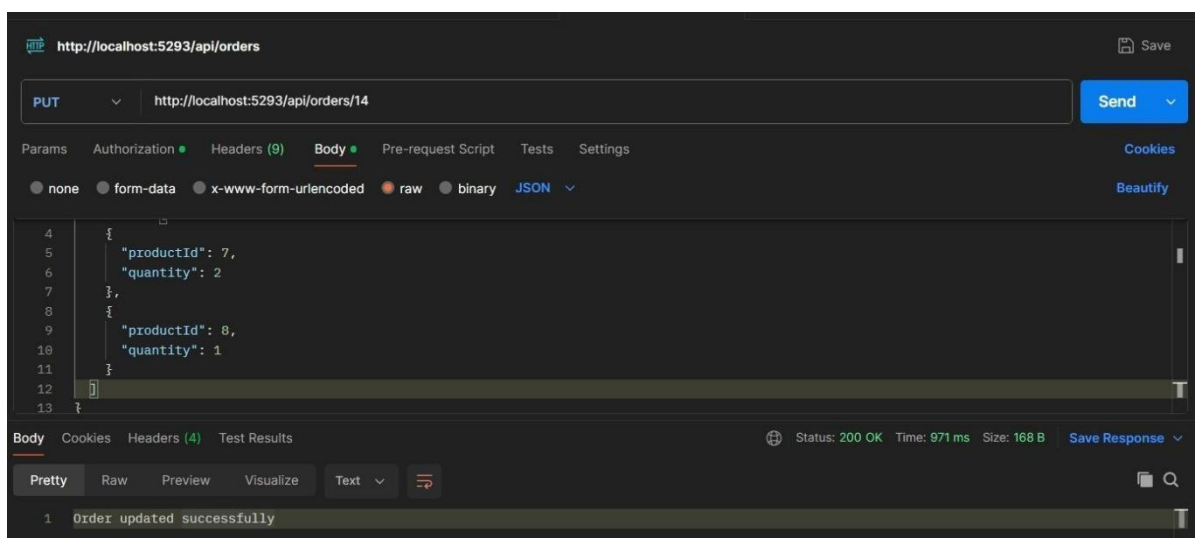
9. View Orders

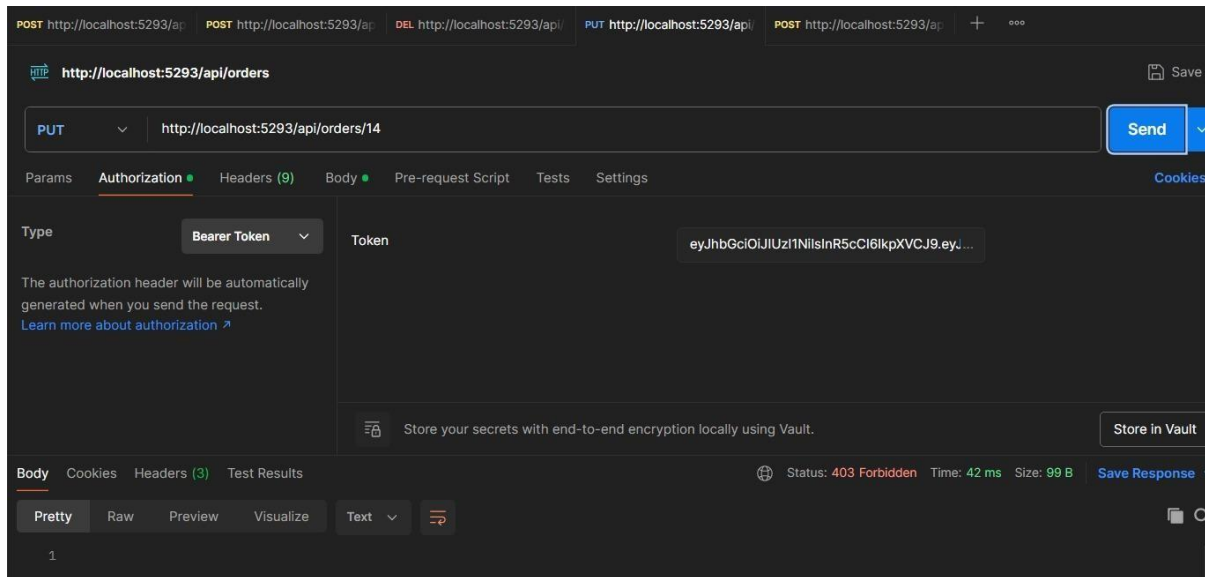


10. View Order by ID

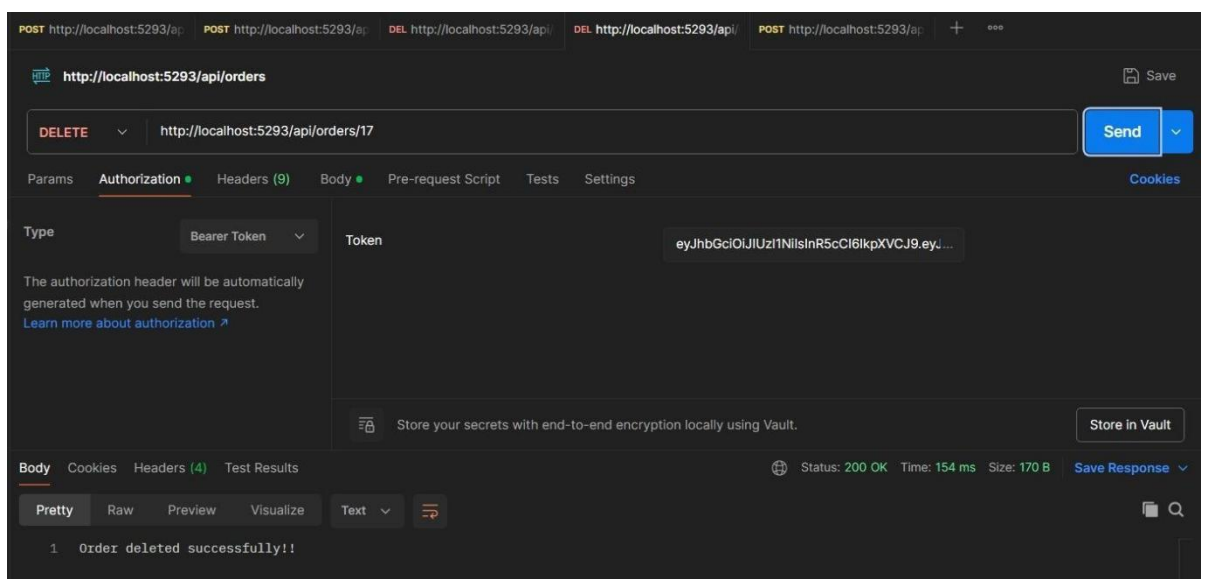
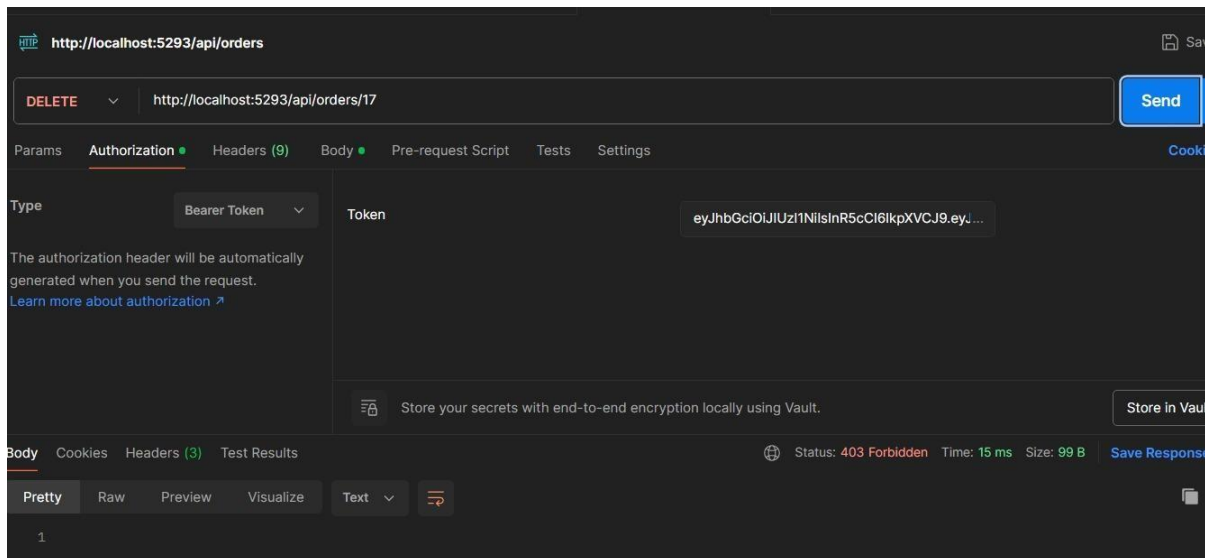


11. Update Order





12. Delete Order



13. conclusion:

This Project is a well-structured, production-ready ASP.NET Core Web API backend that successfully implements the core requirements of a modern e-commerce system. Below is a detailed summary of every key feature, security measure, and design decision implemented in this project.

Security

- **Password Hashing:** User passwords are never stored in plain text. BCrypt.Net-Next is used to hash passwords before saving to the database, and BCrypt.Verify() is used during login to compare the hash — making the system safe even if the database is compromised.
- **JWT Authentication:** All protected endpoints require a valid JWT Bearer token. Tokens are signed using HMAC-SHA256 with a secret key, contain user claims (UserId, Name, Email, Role), and expire after 2 hours.
- **Role-Based Authorization:** Two roles are enforced — Admin and User. Admin-only endpoints (create/update/delete products, manage all orders) are protected with [Authorize(Roles = "Admin")]. Regular users are restricted to their own data only.
- **Token Ownership Enforced on Orders:** The UserId in order creation is extracted directly from the JWT token — not from the request body. This prevents a user from placing orders on behalf of another user.
- **User-Scoped Order Access:** When a User role fetches orders, the API filters results to only return that user's orders. Attempting to access another user's order by ID returns 403 Forbidden.

Validations

- **Input Validation on Registration:** Email and password are checked for null/empty values before processing. Duplicate email registration is rejected with a clear error message.
- **Product Validations:** Name, Description, and ImageUrl cannot be empty. Price must be between 1 and 100,000. Stock cannot be negative. All enforced in both the DTO (data annotations) and the Service layer.
- **Order Item Validations:** Order items list cannot be empty. Quantity must be between 1 and 100. All ProductIds are validated against the database — if any product does not exist, the entire order is rejected.
- **ID Validations:** All endpoints that accept an ID parameter validate that the ID is greater than zero before proceeding, returning 400 Bad Request for invalid values.
- **ModelState Validation:** Controllers check ModelState.IsValid before processing create/update requests, ensuring data annotation rules on DTOs are enforced automatically.

Data Integrity

- **Price Snapshot on Orders:** When an order is placed, the product price is fetched from the database at that moment — not trusted from the client request. This ensures order history reflects the correct price even if the product price changes later.
- **TotalAmount Auto-Calculated:** The order total is always computed server-side as the sum of (price × quantity) for all items. The client cannot manipulate this value.

- Cascade Delete: Deleting a User automatically deletes all their orders. Deleting an Order automatically deletes all its OrderItems. This keeps the database clean with no orphaned records.
- Restrict Delete on Products: A product that exists in any order cannot be deleted (Restrict delete behavior). This protects historical order data from losing product references.
- Unique Email Constraint: A unique index is applied on the User.Email column at the database level, preventing duplicate accounts even under concurrent requests.
- Decimal Precision: Price and TotalAmount columns use decimal(18,2) precision — enforced at the EF model level — preventing floating-point rounding errors in financial data.

Architecture & Code Quality

- Repository Pattern: Data access is fully abstracted behind IProductRepository and IOrderRepository interfaces. Controllers and services never interact with the DbContext directly.
- Service Layer: All business logic lives in dedicated service classes (ProductService, OrderService, AuthService), keeping controllers thin and focused only on HTTP concerns.
- Interface-Driven Design: Every service and repository is registered against its interface (e.g., IOrderService, IProductRepository). This makes the codebase easy to test and swap implementations.
- Scoped Dependency Injection: All services and repositories are registered with Scoped lifetime — a new instance per HTTP request — which is the correct lifetime for EF Core DbContext usage.
- DTO Pattern: Request and response data is shaped through dedicated DTO classes. Database models are never exposed directly in API responses, protecting internal schema details.
- Circular Reference Handling: ReferenceHandler.IgnoreCycles is configured on the JSON serializer, preventing infinite loop errors when serializing related entities (e.g., Order → OrderItems → Order).

API Design & Developer Experience

- Swagger UI: Full OpenAPI documentation is auto-generated and accessible at the root URL. JWT authentication is integrated into Swagger, allowing developers to authorize and test all endpoints directly from the browser.
- CORS Policy: A permissive AllowAll CORS policy is configured, allowing any frontend origin to connect. This is suitable for development and can be tightened for production.
- Clean HTTP Status Codes: Every endpoint returns appropriate status codes — 200 for success, 400 for bad input, 401 for missing auth, 403 for forbidden access, 404 for not found, and 500 for unexpected errors.
- EF Core Migrations: Database schema is managed through 4 tracked EF Core migrations, allowing the database to be recreated or updated reliably across environments using dotnet ef database update.